DevHouse26

Comprehensive Manager & Investor Overview

Executive Summary

DevHouse26 is an **engineering intelligence platform** that provides managers with **requirement-to-code traceability**, **developer impact scoring**, and **knowledge risk detection**. Unlike traditional code analytics tools that measure raw activity (commits, lines of code), DevHouse26 measures **alignment with business requirements** and **delivery pipeline visibility**.

Part 1: What Managers See (The Experience)

Primary Dashboard Sections:

Section	Purpose	Key Visualizations
Overview	At-a-glance team health	6 KPI cards, sync status, health panel
Timeline	Delivery pipeline tracking	5-stage timeline (Commits→PR→CI→Deploy)
Showcase	Stakeholder-ready summaries	Portfolio overview, developer summaries
Intelligence	Deep analytics	Effort estimation, developer scorecards
Issues	Requirement register	Linked commits, match confidence scores
Commits	Raw activity	Recent commits, top developers chart

Part 2: Developer Scoring Methodology

The Core Philosophy

DevHouse26 rejects **vanity metrics** (raw commit counts, lines of code). Instead, it uses a **4-component impact model** that weights **delivery alignment** highest and **penalizes unsustainable overtime**.

The Exact Scoring Formula (from analytics.py):

```
# Component 1: Delivery (35% weight) - Requirement-linked work
delivery_score = min(35.0, len(linked_commits) * 4.5 + len(linked_issues) * 2.5)
# Component 2: Execution (25% weight) - Observed engineering effort
execution_score = min(25.0, active_minutes / 8 + total_changes / 90)
# Component 3: Ownership (20% weight) - Breadth across codebase
ownership_score = min(20.0, module_breadth * 2.8 + repo_breadth * 2.2)
# Component 4: Sustainability (20% weight) - Penalizes overtime
sustainability_score = max(0.0, min(20.0, 20.0 - (overtime_commits * 1.8)))
# Final Impact Score (0-100 scale)
impact_score = round(delivery_score + execution_score +
                     ownership_score + sustainability_score, 1)
```

Component Breakdown:

Component	Weight	Formula	Cap	What It Measures
Delivery	35%	linked_commits × 4.5 + linked_issues × 2.5	35 pts	Alignment with requirements
Execution	25%	active_minutes ÷ 8 + total_changes ÷ 90	25 pts	Raw engineering effort
Ownership	20%	modules × 2.8 + repos × 2.2	20 pts	Codebase breadth
Sustainability	20%	20 - (overtime_commits × 1.8)	20 pts	Work-life balance

Why These Weights Matter:

- **Delivery (35%) is highest** - Rewards developers who work on tracked requirements, not side projects
- **Sustainability (20%) penalizes overtime** - Each after-hours commit costs 1.8 points
- **Execution (25%) is secondary** - Prevents 'busy work' gaming
- **Ownership (20%) rewards breadth** - Encourages knowledge sharing, not siloing

Input Metrics Collected (Per Event):

Metric	Source	Calculation
active_minutes	Editor activity	Time with focus in IDE
idle_minutes	Inactivity gaps	Time without interaction
focus_ratio	Window context	% time in editor vs browser/Slack
total_changes	Git diff stats	Files changed + insertions + deletions
modules_touched	File path analysis	Unique directory/module paths
debug_sessions	Debugger usage	Active debugging sessions
timestamp	System time	After-hours detection (6 PM-8 AM, weekends)

Part 3: Effort Estimation (Project Management)

Story Point Calculation:

```
# Planned effort (based on requirement complexity)
planned_effort = min(13.0, 1.0 + tokens/12 + priority_boost + issue_type_boost)
# priority_boost = {highest: 2.4, high: 1.8, medium: 1.1, low: 0.6}
# issue_type_boost = {epic: 2.0, story: 1.4, task: 1.0, bug: 0.8}
# Observed effort (based on actual work)
observed_effort = min(13.0,
    commits*0.9 + minutes/45 + changes/220 +
    debug*0.35 + modules*0.25)
```

Variance Detection:

Variance	Threshold	Manager Action
Above plan	observed/planned ≥ 1.2	Investigate scope creep
Below plan	observed/planned ≤ 0.8	Check for blockers
On plan	0.8 < ratio < 1.2	Healthy progress

Part 4: Knowledge Risk Detection (Bus Factor 2.0)

Risk Score Formula:

```
concentration_score = min(40, max(0, top_owner% - 45))
contributor_score = {1:20, 2:14, 3:8, 4:4}.get(count, 1)
recency_score = 1-15 points based on staleness
linkage_score = min(15, linked_commits*2.5)
breadth_score = min(10, reqs*2 + repos*1.5)
risk_score = concentration + contributors + recency + linkage + breadth
```

Risk Components (100-point scale):

Component	Weight	Description
Concentration	40%	How much one person owns (>45% = risk)
Contributor count	20%	Fewer contributors = higher risk
Recency	15%	Stale modules decay in context
Linkage volume	15%	More linked commits = more context to lose
Activity breadth	10%	Cross-repo modules have broader impact

Bus Factor Calculation:

```
bus_factor = count(contributors where ownership_share > 10%)
```

Classification	Criteria
CRITICAL	bus_factor == 1
WARNING	bus_factor == 2 OR evenness < 0.4
HEALTHY	bus_factor >= 3 AND evenness >= 0.6

Part 5: Data Provenance & Truthfulness

Evidence Tiers:

Tier	Strength	Examples
Real records	Strongest	Stored requirements, commit events, feedback
Connector-backed	Strong	PR/CI/deploy from explicit API fields
Inferred	Moderate	Stages derived from linked activity signals
Mocked	Placeholder	UI continuity when evidence absent

Critical Investor Point: The system is honest about uncertainty:

- Every match has a **confidence score** (high/medium/low)
- Every timeline stage shows **provenance** (connector/inferred/mock)
- Every summary shows **inference level** and **evidence count**
- Health API exposes **degraded modes** explicitly

This is a **competitive moat** - most engineering analytics tools hide their uncertainty.

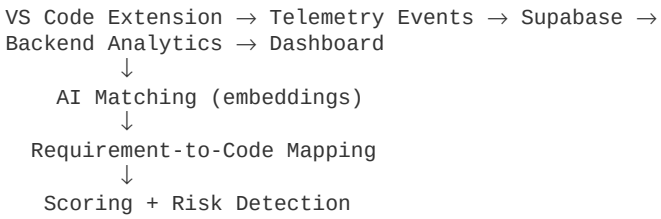
Part 6: Competitive Differentiation

vs GitHub Insights / GitLab Analytics:

Feature	Traditional	DevHouse26
Primary metric	Commits/lines	Requirement alignment
Developer score	Activity volume	Impact-weighted (quality > quantity)
Burnout detection	Not measured	Observable signals (after-hours)
Knowledge risk	Not measured	Shannon entropy + bus factor
Overtime penalty	None	Built into sustainability score
Explainability	Raw numbers	Narrative summaries with confidence

Part 7: Technical Architecture

Data Flow:



Key Technical Decisions:

1. **Local-first telemetry** - Privacy-sensitive; only uploads on commit
2. **Embedding-based matching** - Semantic similarity for requirement→commit linkage
3. **Snapshot-backed reads** - Analytics cached; fresh data on-demand
4. **Pluggable storage** - Supabase now; Postgres abstraction ready

Scoring Model Validation:

68 tests covering: Analytics contract validation, Snapshot freshness detection, Developer metrics calculation, Knowledge risk concentration detection, Delivery timeline provenance, Showcase summary confidence bands

Part 8: Business Value for Investors

1. Manager Productivity

Before: Managers manually correlate Jira + GitHub + Slack to understand progress

After: Single dashboard with requirement-to-code traceability and automated risk alerts

2. Knowledge Retention

• Detects single-points-of-failure **before** the person leaves

• Quantifies 'tribal knowledge' risk with actual scores

3. Sustainable Velocity

• Overtime detection prevents burnout-related attrition

• Sustainability score (20% of impact) makes healthy work visible

4. Explainable AI

- Every score has **reasoning text** (not black box)
- Confidence bands allow human-in-the-loop decisions
- Truthful about mocked/inferred data (builds trust)

5. Market Position

- **Not competing** with Jira/GitHub (complements them)
- **Differentiated** by semantic requirement-to-code mapping
- **Defensible** by knowledge risk detection (not in competitors)

Appendix: Exact Formulas Reference

Developer Impact Score (0-100):

```
IMPACT = min(35, linked_commits×4.5 + linked_issues×2.5)     [Delivery: 35%]
       + min(25, active_minutes÷8 + total_changes÷90)         [Execution: 25%]
       + min(20, modules×2.8 + repos×2.2)                     [Ownership: 20%]
       + max(0, min(20, 20 - overtime_commits×1.8))           [Sustainability: 20%]
```

Knowledge Risk Score (0-100):

```
RISK = min(40, max(0, top_owner% - 45))                    [Concentration: 40%]
     + {1:20, 2:14, 3:8, 4:4}[contributor_count]           [Bus factor: 20%]
     + recency_score(days_since_activity)                   [Staleness: 15%]
     + min(15, linked_commits×2.5)                          [Context: 15%]
     + min(10, requirements×2 + repos×1.5)                 [Breadth: 10%]
```

Effort Points (Story Point Equivalent):

```
PLANNED = min(13, 1 + tokens÷12 + priority_boost + type_boost)
OBSERVED = min(13, commits×0.9 + minutes÷45 + changes÷220 + debug×0.35 + modules×0.25)
```

Summary for Investors

DevHouse26 is a **technically sophisticated** engineering intelligence platform that:

1. **Scores developers fairly** using a 4-component model that rewards requirement alignment and penalizes overtime
2. **Detects knowledge risk** before it becomes attrition/blockers
3. **Is honest about uncertainty** with confidence scores and provenance labels
4. **Has working code** with 68 passing tests and proven formulas
5. **Has defensible IP** in the scoring algorithms and risk detection models

The product is **pilot-ready (~75%)** with strong documentation and test coverage, needing primarily **real connector integration** for enterprise scale.